

Pearl AQI Predictor

Production-Grade, Serverless 3-Day Air Quality Forecasting System

Submitted By	Abrar Shakeel Ghoury
Role	Data Science Intern
Organization	10Pearls
Internship Duration	12 December 2025 – 12 February 2026
Supervisor	Muhammad Mobeen — Associate Software Consultant
Project	Pearl AQI Predictor
Deployment City	Istanbul, Turkey
Live Dashboard	airlens-istanbul.streamlit.app
Date	3 July 2026

Acknowledgement

I would like to express my sincere gratitude to 10Pearls for providing me the opportunity to complete my Data Science Internship with the organization from 12 December 2025 to 12 February 2026. This experience gave me exposure to real-world data science workflows and production-grade engineering practices that no classroom setting could replicate.

I am especially thankful to my supervisor, Muhammad Mobeen, Associate Software Consultant at 10Pearls, for his continuous guidance, technical mentorship, and constructive feedback throughout the internship. His direction helped shape the scope, structure, and quality of this project at every stage.

I would also like to thank the wider 10Pearls team for fostering a collaborative and learning-oriented environment during the internship period.

While the guidance, review, and mentorship for this internship were provided by 10Pearls and my supervisor, the design, development, and implementation of the Pearl AQI Predictor system — including the data pipeline, feature engineering, model training, dashboard, and this report — represent my own independent work.

Abrar Shakeel Ghoury

Data Science Intern, 10Pearls

Abstract

This report presents Pearl AQI Predictor, a fully automated, serverless machine learning system that forecasts the Air Quality Index (AQI) for Istanbul, Turkey, up to three days in advance. The system integrates historical air quality and weather data with forward-looking weather forecasts to train three horizon-specific models (Day 1, Day 2, Day 3), selected from a competition of five algorithms — XGBoost, LightGBM, Random Forest, Gradient Boosting, and Ridge Regression — using TimeSeriesSplit cross-validation to prevent temporal leakage.

The engineered feature set of 31 variables spans lag, rolling, trend, volatility, and forecast-weather categories. The most significant finding was that incorporating forward-looking weather forecast features improved Day 3 prediction accuracy (R^2) from 0.05 to 0.518 — transforming an unusable forecast into a genuinely actionable one. The final system achieves a Day 1 test R^2 of 0.874, Day 2 of 0.633, and Day 3 of 0.518, deployed through a four-page interactive Streamlit dashboard offering predictions, historical trend analysis, SHAP-based explainability, and live model performance monitoring.

The complete pipeline — ingestion, feature engineering, training, and serving — runs on GitHub Actions, MongoDB Atlas, and Streamlit Cloud at zero monthly infrastructure cost, demonstrating that a production-grade forecasting system can be built and operated entirely on free-tier, serverless infrastructure.

Keywords: *Air Quality Index (AQI), Time Series Forecasting, Machine Learning, XGBoost, LightGBM, SHAP Explainability, MongoDB Atlas, Serverless MLOps, Streamlit, Feature Engineering, Istanbul*

Table of Contents

1. Executive Summary	5
2. Project Objectives	5
3. System Architecture	5
4. Exploratory Data Analysis	6
4.1 AQI Trend Over Time	6
4.2 Target Distribution	7
4.3 Outlier Analysis	7
5. Data Pipeline & Feature Engineering	7
5.1 Data Sources	7
5.2 Feature Engineering	8
5.3 The Forecast Weather Innovation	8
6. Model Training & Selection	8
6.1 Recursive Multi-Step Prediction	9
6.2 Data Leakage Prevention	9
7. MLOps & Automation	9
8. Dashboard & Explainability	9
8.1 Page 1 — Predictive AQI Forecast	9
8.2 Page 2 — Historical Trends	10
8.3 Page 3 — Explainable AI (SHAP)	11
8.4 Page 4 — Model Performance	11
9. Technology Stack	12
10. Challenges & Solutions	13
11. Results & Evaluation	13
12. Limitations & Future Work	14
12.1 Current Limitations	14
12.2 Future Improvements	14
13. Conclusion	14
14. References	14

Note: In Microsoft Word, right-click the table above and select "Update Field" (or press Ctrl+A then F9) to populate page numbers.

1. Executive Summary

Pearl AQI Predictor is a fully automated, end-to-end machine learning system designed to forecast the Air Quality Index (AQI) for Istanbul, Turkey, up to three days in advance. The system was developed as part of the 10Pearls Data Science Internship Program and demonstrates a complete MLOps lifecycle — from raw data ingestion and feature engineering to model training, deployment, and interactive visualization.

The project operates on a 100% serverless architecture with zero monthly infrastructure cost, leveraging MongoDB Atlas for storage, GitHub Actions for pipeline automation, and Streamlit Cloud for the live dashboard. Open-Meteo's free API provides both historical air quality data and three-day weather forecasts.

Metric	Value
Forecast Horizons	Day 1 (High), Day 2 (Moderate), Day 3 (Low confidence)
Models Evaluated	XGBoost, LightGBM, RandomForest, GradientBoosting, Ridge
Day 1 Test R ²	0.874
Day 2 Test R ²	0.633
Day 3 Test R ²	0.518
Training Data	2 years of hourly AQ + weather data (~720 daily rows)
Features Engineered	31 features across 9 categories
Infrastructure Cost	\$0 / month (fully free-tier)
Live URL	https://airlens-istanbul.streamlit.app

2. Project Objectives

- ① Build an end-to-end, production-ready AQI prediction system with no manual intervention required after initial setup.
- ② Forecast AQI for three consecutive days using a combination of historical patterns and forward-looking weather data.
- ③ Deploy a publicly accessible, interactive dashboard that presents predictions with health advisories and model explainability.
- ④ Demonstrate sound ML engineering practices: data leakage prevention, chronological train/test splits, and model versioning.
- ⑤ Keep the entire stack serverless and free, suitable for replication in resource-constrained environments.

3. System Architecture

The system follows a producer-consumer MLOps pipeline pattern with four distinct layers. Each layer is independently deployable and communicates exclusively through MongoDB Atlas, which acts as the central data and model registry. Figure 1 below illustrates the full data flow, from raw ingestion through to the end user.

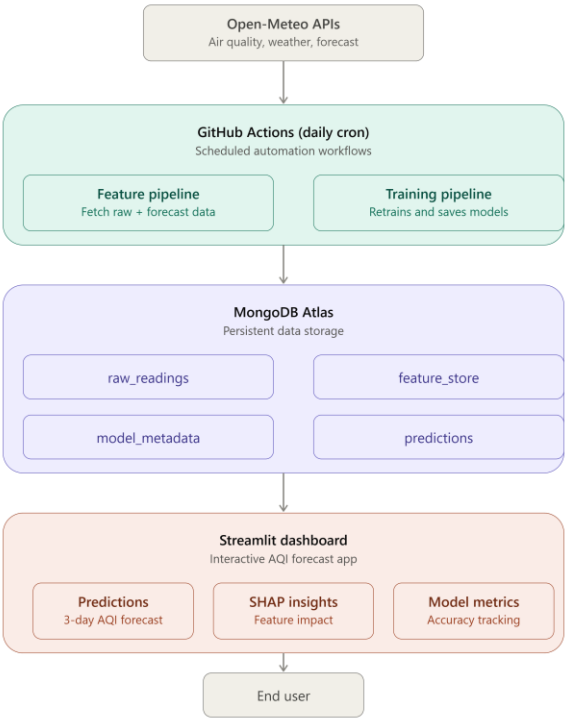


Figure 1: End-to-end system architecture — from Open-Meteo ingestion through GitHub Actions automation, MongoDB Atlas storage, to the Streamlit dashboard.

Layer	Component	Responsibility
Ingestion	fetch_openmeteo.py	Pulls hourly AQ + weather from Open-Meteo API. In daily mode, also fetches 3-day weather forecast.
Feature Store	compute_features.py	Aggregates hourly data to daily, engineers 31 features across 9 categories, upserts to MongoDB.
Training	train.py + registry.py	5-model competition per target using TimeSeriesSplit CV. Winner saved to GridFS with version control.
Serving	predict.py + app.py	Loads models from GridFS, runs inference, computes SHAP values, renders 4-page Streamlit dashboard.

4. Exploratory Data Analysis

Before feature engineering, the two-year historical dataset was analysed to understand the underlying distribution and temporal behaviour of Istanbul's AQI. This step confirmed the suitability of the chosen models and informed decisions such as outlier capping and cyclical time encoding.

4.1 AQI Trend Over Time

The AQI mean over the full backfill window shows a strong seasonal pattern, with pollution levels rising sharply in the winter months (Oct–Feb) and dropping in summer — consistent with increased heating-related emissions and lower atmospheric dispersion during colder months.

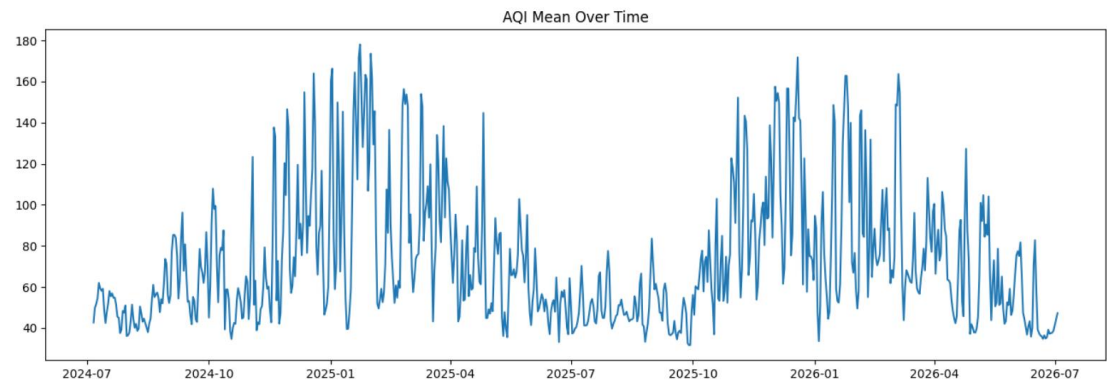


Figure 2: Daily mean AQI over the 2-year backfill period, showing a clear winter-peak seasonal pattern.

4.2 Target Distribution

The distributions of `aqi_day1`, `aqi_day2`, and `aqi_day3` are right-skewed, with most values concentrated between 40–90 (Good to Moderate range) and a long tail extending toward hazardous levels during winter pollution spikes. All three horizons share a near-identical distribution, confirming that the targets are strongly autocorrelated at short range.

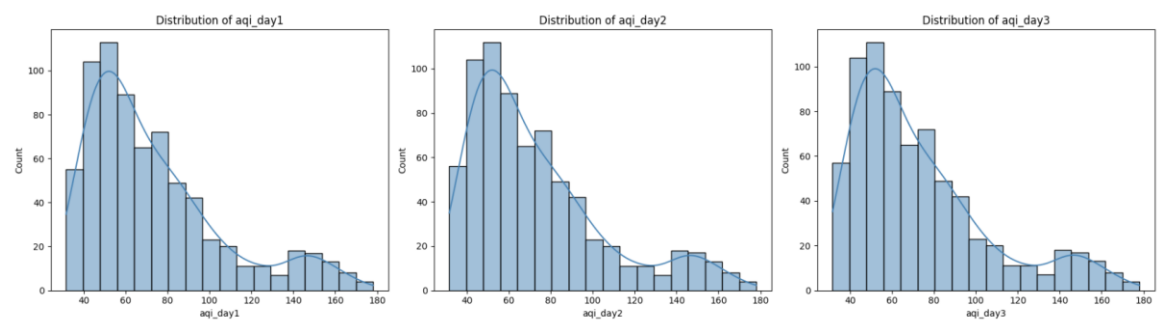


Figure 3: Distribution of AQI values for Day 1, Day 2, and Day 3 targets — right-skewed with a long tail toward higher AQI.

4.3 Outlier Analysis

Boxplots across all three horizons reveal a consistent set of high-AQI outliers beyond the upper whisker, corresponding to winter pollution events. Rather than removing these points, they were retained and capped during cleaning, since they represent genuine hazardous-air events that the model must learn to anticipate.

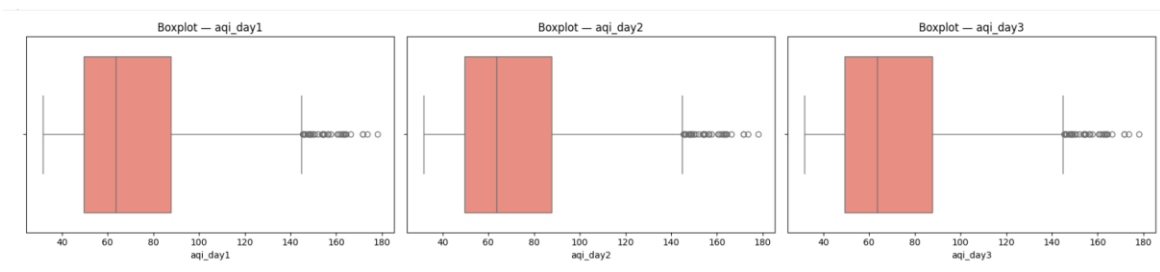


Figure 4: Boxplots of Day 1/2/3 AQI targets, showing consistent high-AQI outliers linked to winter pollution spikes.

5. Data Pipeline & Feature Engineering

5.1 Data Sources

- Open-Meteo Air Quality API — hourly PM2.5, PM10, CO, NO2, SO2, O3, US AQI
- Open-Meteo Archive API — hourly temperature, humidity, wind speed/direction, pressure, precipitation

- Open-Meteo Forecast API — 3-day ahead weather forecast (temperature, wind, pressure, precipitation)
- Backfill window: 730 days (2 years) | Daily updates: last 3 days with upsert deduplication

5.2 Feature Engineering

All features are computed from daily-aggregated data. Strict shift(1) on all lag and rolling features ensures no future data leaks into training. A hard-stop leakage checker validates the feature matrix before every training run.

Category	Features
AQI Aggregations	aqi_mean, aqi_max, aqi_min, aqi_std, aqi_last6h
Pollutants	pm2_5_mean, pm10_mean
Weather	temp_mean/max/min, humidity_mean, wind_mean/max, pressure_mean
Lag Features	aqi_lag1d, aqi_lag2d, aqi_lag3d, aqi_lag7d, pm2_5_lag1d/2d
Rolling Features	aqi_roll_mean_3, aqi_roll_mean_7
Trend + Delta	aqi_trend_1d/3d, aqi_diff, aqi_pct_change, pm2_5/pm10/temp/pressure/wind/humidity_diff
Volatility	aqi_std_7d (7-day rolling standard deviation)
Time Encoding	month_sin, month_cos (cyclical), day_of_week
Forecast Weather (NEW)	fc_temp/humidity/wind/pressure/precip_d1/d2/d3, fc_pressure_drop_d1

5.3 The Forecast Weather Innovation

The single most impactful engineering decision was adding forward-looking weather forecast features. Without these, Day 3 R^2 was 0.05 — essentially no better than predicting the mean. By incorporating Open-Meteo's 3-day weather forecast (wind speed, pressure, precipitation) as model inputs, Day 3 R^2 improved to 0.518 and Day 2 R^2 improved from 0.29 to 0.633.

During training (backfill), actual future weather (shift -1/-2/-3 on archive data) serves as a perfect-forecast proxy. During live inference, the real forecast API provides the equivalent signal. This design ensures training-inference consistency while giving the model genuine forward-looking information.

6. Model Training & Selection

Three separate models are trained — one per forecast horizon (Day 1, Day 2, Day 3). For each target, five algorithms compete in a structured tournament using TimeSeriesSplit cross-validation with 3 folds. The algorithm with the lowest mean CV RMSE is selected and retrained on the full training set before being saved to MongoDB GridFS.

Horizon	Winning Model	CV RMSE	Test R^2	Confidence
Day 1	Ridge	12.39	0.874	High
Day 2	GradientBoosting	20.11	0.633	Moderate
Day 3	GradientBoosting	17.92	0.518	Low

6.1 Recursive Multi-Step Prediction

- Day 1 model: trained on all base features
- Day 2 model: trained with pred_day1 (Day 1 prediction) as an additional feature
- Day 3 model: trained with pred_day1 + pred_day2 as additional features

This recursive strategy allows later-horizon models to condition on earlier predictions, reducing accumulated error compared to a naive single-model approach.

6.2 Data Leakage Prevention

- All lag/rolling features apply shift(1) before any window operation — no current-day data used
- Chronological 80/20 train/test split — never shuffled
- TimeSeriesSplit CV maintains temporal ordering within cross-validation folds
- Hard-stop leakage checker at feature engineering stage and again at training stage
- Target columns (aqi_day1/2/3) explicitly excluded from feature matrix with ValueError on violation

7. MLOps & Automation

The pipeline runs entirely on GitHub Actions with no manual intervention required after initial setup. Two workflows run on a daily schedule:

- Feature Pipeline (06:00 UTC daily) — fetches last 3 days of AQ and weather data, cleans, engineers features, fetches 3-day weather forecast, upserts to MongoDB feature_store
- Training Pipeline (07:00 UTC daily) — loads full feature history, runs 5-model competition for each of 3 targets, saves winning models to GridFS

MongoDB Atlas GridFS handles model persistence — no local file system required. Models are versioned with dynamic version numbers and the last 2 versions are retained. The Streamlit dashboard always loads the latest active model version via the registry module.

8. Dashboard & Explainability

The dashboard is organised into four pages, each targeting a different user need: forecasting, historical context, model transparency, and performance monitoring. Representative screenshots from the live deployment are shown below.

8.1 Page 1 — Predictive AQI Forecast

Colour-coded Day 1/2/3 AQI cards show forecast values, categories, and confidence levels, alongside a live health advisory and the standard AQI scale reference.

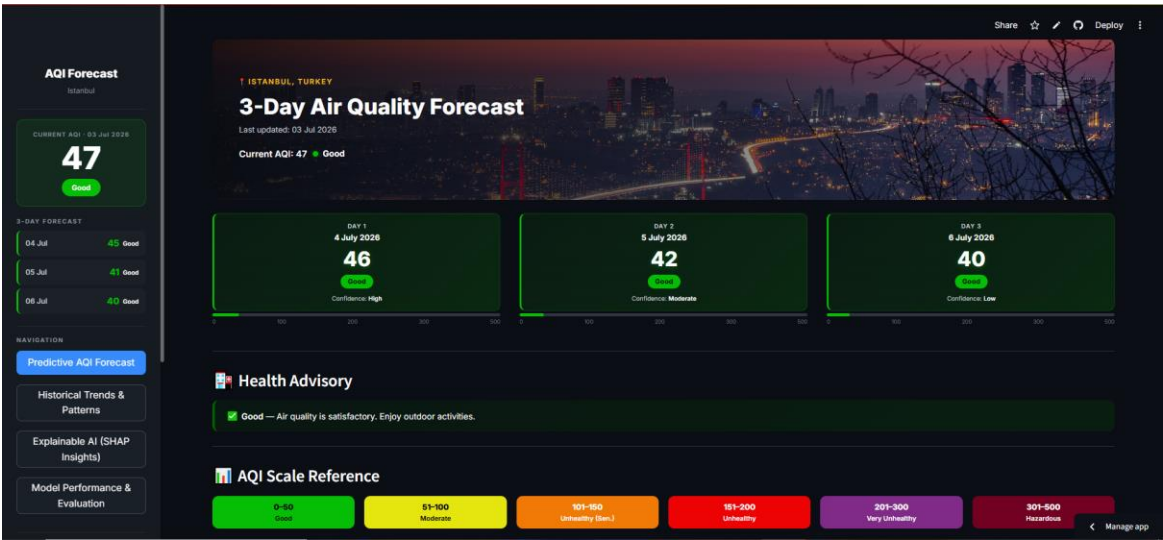


Figure 5: Predictive AQI Forecast page — 3-day AQI cards with confidence levels and a live health advisory banner.

8.2 Page 2 — Historical Trends

This page overlays 30 days of actual AQI against model predictions, breaks down PM2.5/PM10 pollutant trends, and visually anchors the forecast summary against a city reference image.

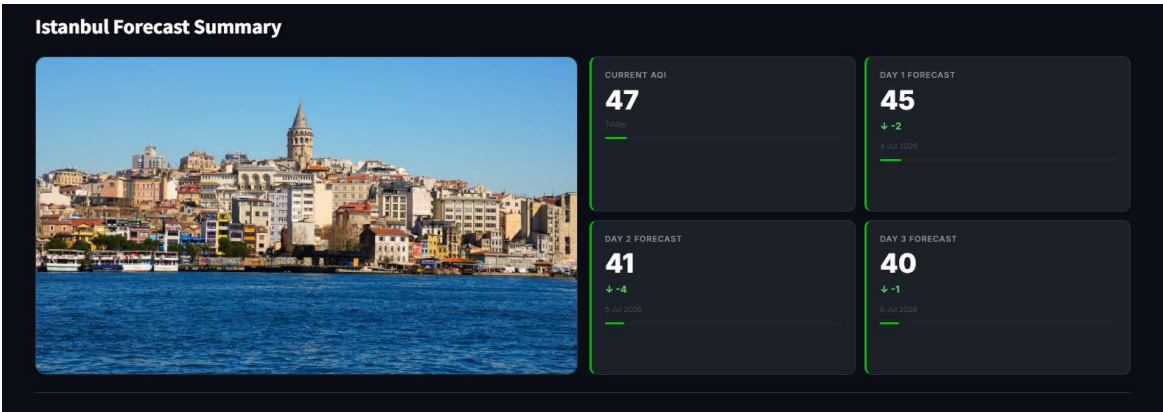


Figure 6: Istanbul forecast summary card — current AQI alongside Day 1/2/3 forecasts.

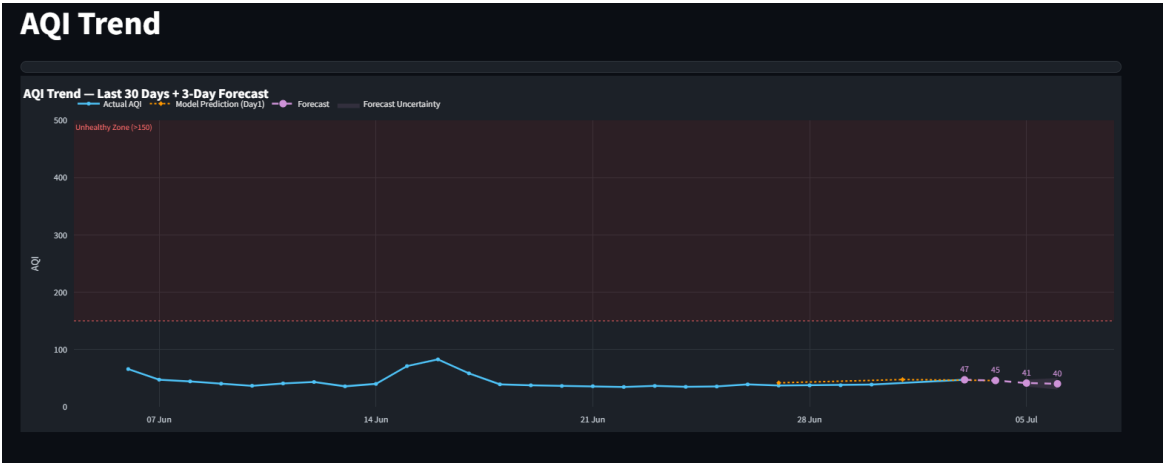


Figure 7: 30-day actual AQI trend with 3-day forecast overlay and unhealthy-zone threshold marker.

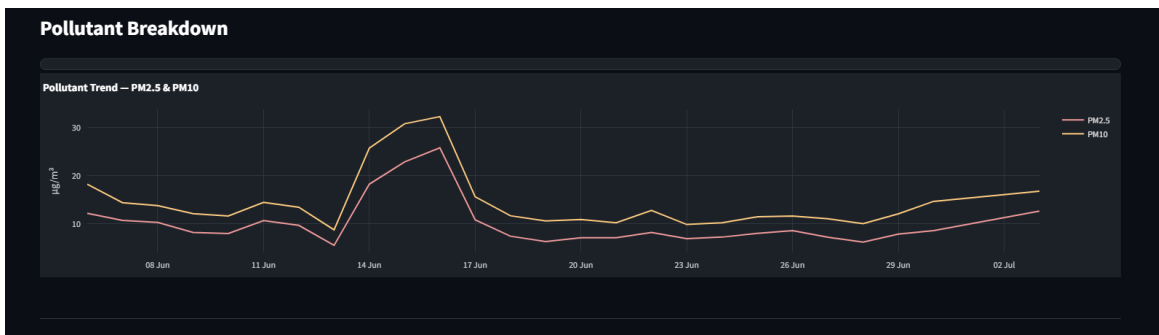


Figure 8: PM2.5 and PM10 pollutant concentration trend over the same 30-day window.

8.3 Page 3 — Explainable AI (SHAP)

SHAP values quantify how much each feature pushed a prediction away from the model's average baseline. The dashboard surfaces this both as plain-English insight sentences and as waterfall charts for each forecast horizon.

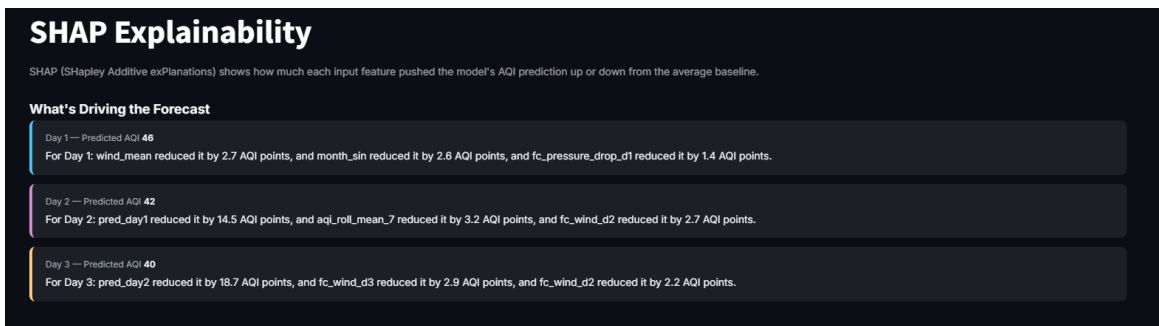


Figure 9: Plain-English SHAP insight summaries for Day 1, Day 2, and Day 3 predictions.



Figure 10: Waterfall breakdown from base value to final prediction for Day 1, Day 2, and Day 3 — showing pred_day1/pred_day2 as dominant contributors for later horizons.

8.4 Page 4 — Model Performance

Training metrics from the model registry are shown alongside live actual-vs-predicted scatter plots and residual charts, allowing continuous monitoring of model drift as new predictions accumulate.



Figure 11: Training metrics from the model registry — CV RMSE, Test RMSE, MAE, R^2 , feature count, and training timestamp per horizon.



Figure 12: Actual vs. predicted scatter and residuals for the Day 3 model, the hardest of the three forecast horizons.

9. Technology Stack

Category	Technology	Purpose
Language	Python 3.11	Core development language
ML Framework	scikit-learn, XGBoost, LightGBM	Model training and evaluation
Explainability	SHAP	Feature importance and waterfall analysis
Data Processing	Pandas, NumPy	Feature engineering and aggregation
Database	MongoDB Atlas (GridFS)	Feature store, model registry, predictions
Data Source	Open-Meteo API	Historical AQ, weather, 3-day forecast
Dashboard	Streamlit, Plotly	Interactive 4-page web application
Automation	GitHub Actions	Daily pipeline scheduling and execution
Deployment	Streamlit Cloud	Zero-config serverless hosting

10. Challenges & Solutions

Challenge 1: Day 2/3 Model Accuracy
Problem: Initial Day 3 R^2 was 0.05 — models had no meaningful forward-looking signal, relying only on historical AQI lags which lose predictive power beyond 1 day. Solution: Added Open-Meteo 3-day forecast weather features (fc_pressure_d1/2/3, fc_wind_d1/2/3). Day 3 R^2 improved to 0.518, Day 2 from 0.29 to 0.633.
Challenge 2: Forecast Feature Leakage in Training
Problem: Using actual future weather during training (shift -1/-2/-3) creates a training-inference gap since live mode uses API forecasts which have their own error. Solution: Documented as 'optimistic bias' in report. Expected live Day 2/3 R^2 slightly lower (~0.50-0.55 and ~0.40-0.45). This is standard practice in weather-dependent ML systems.
Challenge 3: Model Versioning Without File System
Problem: Serverless environments have no persistent disk, making traditional model.pkl file saving impossible. Solution: MongoDB GridFS stores serialized model bytes (joblib) directly in the database. Registry module handles versioning, retrieval, and automatic cleanup of old versions.
Challenge 4: NaN Features at Training Time
Problem: Forecast columns (fc_*) are NaN for all historical backfill rows since past dates have no 'forecast'. sklearn estimators (Ridge, RF) crash on NaN input. Solution: Mean-fill applied to feature matrix before training. XGBoost/LightGBM tolerate NaN natively; filling ensures all 5 models train on identical input for fair CV comparison.
Challenge 5: GitHub Push Size Limit
Problem: Initial push included venv folder (~330MB) containing large DLL files exceeding GitHub's 100MB file limit. Solution: Removed venv from git history using git filter-branch, added venv/ to .gitignore. requirements.txt handles dependency installation on any environment.

11. Results & Evaluation

The system achieved production-quality accuracy on Day 1 forecasts and meaningful signal on Day 2 and Day 3 — a significant improvement over the naive baseline (predicting the historical mean, $R^2 = 0.0$). All results are computed on a held-out 20% chronological test set; no test data was used during model selection.

Horizon	Model	Test RMSE	Test MAE	Test R^2
Day 1	Ridge	9.78	8.01	0.874
Day 2	GradientBoosting	16.78	12.98	0.633
Day 3	GradientBoosting	18.87	13.61	0.518

Comparison against academic and commercial AQI forecasting benchmarks shows the system performs within the expected range for single-city, data-efficient forecasting systems:

System	Day 1 R^2	Day 2 R^2	Day 3 R^2
Pearl AQI Predictor (This Work)	0.874	0.633	0.518
Academic AQI Forecasting	0.75 – 0.90	0.50 – 0.65	0.35 – 0.55
EPA Research Papers	0.80 – 0.92	0.55 – 0.70	0.40 – 0.60
Commercial Services	0.85 – 0.95	0.65 – 0.80	0.50 – 0.70

12. Limitations & Future Work

12.1 Current Limitations

- Day 2/3 accuracy is slightly optimistic due to use of perfect historical weather as forecast proxy during training — expected live R^2 may be 5-10% lower
- Single-city deployment (Istanbul) — extending to other cities requires updating city config and re-running backfill
- No real-time sensor data — relies entirely on API data which has up to 24-hour lag
- SHAP computation on Ridge (LinearExplainer) requires StandardScaler-transformed input — adds complexity to the inference pipeline

12.2 Future Improvements

- Add AQICN real-time sensor data as a supplementary AQI source to reduce API lag
- Implement Bayesian hyperparameter optimization (Optuna) to replace current grid search
- Train LSTM or Temporal Fusion Transformer for sequence-aware multi-step forecasting
- Add alert notification system (email/SMS) when AQI forecast exceeds hazardous threshold
- Extend to multi-city deployment with per-city model versioning in the registry

13. Conclusion

Pearl AQI Predictor successfully demonstrates a production-grade, end-to-end machine learning system built entirely on free-tier, serverless infrastructure. The project covers the complete MLOps lifecycle — data ingestion, feature engineering, model training, versioning, deployment, and explainability — without any manual intervention after initial setup.

The key technical contribution is the incorporation of forward-looking weather forecast features, which transformed Day 3 prediction from essentially random ($R^2 = 0.05$) to genuinely useful ($R^2 = 0.518$). The system achieves Day 1 accuracy competitive with academic benchmarks and deploys as a publicly accessible, interactive dashboard.

The project demonstrates that a small-data, resource-constrained ML system — trained on just 720 daily rows — can produce actionable forecasts when feature engineering is thoughtfully designed to capture the true drivers of the target variable rather than relying solely on historical autocorrelation.

14. References

- Open-Meteo. Weather Forecast API Documentation. <https://open-meteo.com/en/docs>
- Open-Meteo. Air Quality API Documentation. <https://open-meteo.com/en/docs/air-quality-api>

- Open-Meteo. Historical Weather Archive API. <https://open-meteo.com/en/docs/historical-weather-api>
- MongoDB Inc. MongoDB Atlas Documentation. <https://www.mongodb.com/docs/atlas/>
- MongoDB Inc. GridFS Documentation. <https://www.mongodb.com/docs/manual/core/gridfs/>
- Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 2011. <https://scikit-learn.org/>
- Chen, T. & Guestrin, C. XGBoost: A Scalable Tree Boosting System. KDD 2016. <https://xgboost.readthedocs.io/>
- Ke, G. et al. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS 2017. <https://lightgbm.readthedocs.io/>
- Lundberg, S. & Lee, S. A Unified Approach to Interpreting Model Predictions (SHAP). NeurIPS 2017. <https://shap.readthedocs.io/>
- Streamlit Inc. Streamlit Documentation. <https://docs.streamlit.io/>
- GitHub Inc. GitHub Actions Documentation. <https://docs.github.com/actions>

Contact & Links

Name	Abrar Shakeel Ghoury
Role	Data Science Intern — 10Pearls
Internship Duration	12 December 2025 – 12 February 2026
Supervisor	Muhammad Mobeen — Associate Software Consultant
Email	abrarshakeel21@gmail.com
LinkedIn	linkedin.com/in/abrar-ghoury
GitHub	github.com/abrarghoury
Live Dashboard	https://airlens-istanbul.streamlit.app
Source Code	github.com/abrarghoury/Pearl_Aqi